Chapter 5 Lab – Pre-trained Language Models, BERT, DistilBERT, and Text Classification

Advanced Machine Learning for Finance

Giovanni Della Lunga

Master's Degree in Quantitative Finance – University of Bologna

Bologna – 2026

Learning objectives

At the end of this notebook, students should be able to:

- explain the idea of pre-trained language models and transfer learning;
- understand why BERT is an encoder-based Transformer model for language understanding;
- use Hugging Face tokenizers to prepare text inputs for BERT-like models;
- run sentiment inference with a pre-trained DistilBERT classifier;
- build a PyTorch dataset, classifier, training loop, and evaluation function for news classification.

Notebook structure

The notebook develops the topic in two main parts.

Part 1: using pre-trained models

- pre-trained language models;
- transfer learning;
- BERT and tokenization;
- DistilBERT sentiment analysis;
- Hugging Face `pipeline`.

Part 2: fine-tuning for classification

- BBC News dataset;
- text preprocessing;
- PyTorch `Dataset` and `DataLoader`;
- custom BERT classifier;
- training, validation, and test evaluation.

What is a pre-trained language model?

A pre-trained language model is a neural network first trained on a very large text corpus, before being adapted to a specific downstream task.

The main idea is that the model learns general linguistic regularities:

- syntactic patterns;
- semantic relations;
- contextual word usage;
- statistical regularities of natural language.

Once learned, these representations can be reused for tasks such as sentiment analysis, named entity recognition, text classification, question answering, and summarization.

Why pre-training matters

Training a modern language model from scratch is expensive because it requires:

- a very large corpus;
- substantial computational resources;
- long training time;
- careful optimization and engineering.

Using a pre-trained model allows us to start from a network that already contains a rich representation of language. The downstream model does not learn language from zero; it learns how to use linguistic representations for a specific task.

Transfer learning in NLP

Transfer learning means that knowledge acquired while solving one task is reused for another related task.

In NLP, the usual workflow is:

1. **Pre-training**: train a large model on a broad language objective.
2. **Feature extraction**: use the model to produce contextual embeddings.
3. **Fine-tuning**: add a task-specific head and train on the target dataset.

Fine-tuning may update all model parameters, or only the parameters of the task-specific layers.

Freezing versus fine-tuning

Frozen encoder

- BERT weights are kept fixed.
- Only the classification head is trained.
- Training is faster and cheaper.
- Useful when little data is available.

Full fine-tuning

- Both BERT and the classification head are updated.
- The model adapts more deeply to the target task.
- Usually gives better performance.
- Requires more care to avoid overfitting.

BERT: Bidirectional Encoder Representations from Transformers

BERT is an encoder-based Transformer architecture introduced for language understanding tasks.

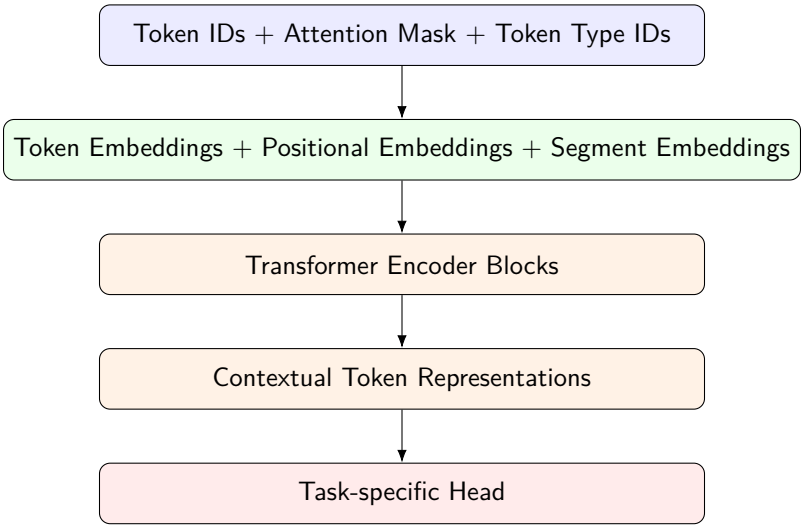
Its key feature is **bidirectionality**: each token representation is computed using both left and right context.

For example, in the sentence:

The bank approved the loan.

the representation of the word *bank* depends on the surrounding words *approved* and *loan*, not only on the words before it.

BERT architecture at a high level



BERT base and BERT large

Model	Encoder layers	Attention heads	Hidden size
BERT base	12	12	768
BERT large	24	16	1024

The notebook uses BERT mainly as a conceptual foundation, then moves to DistilBERT for a sentiment analysis example and to `bert-base-cased` for the custom news classifier.

First tokenization example

The notebook starts by using the Hugging Face BERT tokenizer.

```
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
text = "BERT preprocessing is essential."
tokens = tokenizer.tokenize(text)

print(tokens)
```

The tokenizer converts raw text into subword tokens that can be mapped into integer IDs and passed to the model.

What the tokenizer produces

A BERT tokenizer usually returns a dictionary containing several tensors.

Key	Meaning
`input_ids`	Integer IDs representing tokens.
`attention_mask`	Indicates real tokens versus padding tokens.
`token_type_ids`	Indicates sentence A versus sentence B when needed.

These tensors are the actual numerical input consumed by BERT-like models.

Special tokens

BERT adds special tokens to the original text.

- `[CLS]`: placed at the beginning of the sequence. Its final representation is often used for classification.
- `[SEP]`: marks the end of a sequence, or separates two sequences.
- `[PAD]`: fills shorter sequences so that all examples in a batch have the same length.

For a single-sequence classification task, the model can use the contextual representation of `[CLS]` as a compact summary of the whole input sentence.

Attention mask

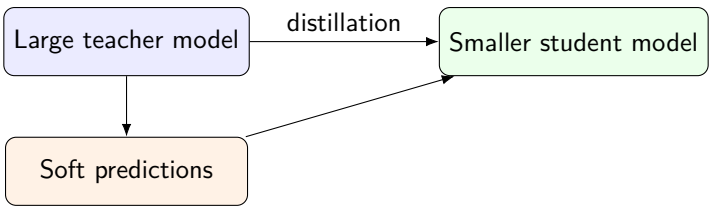
The attention mask tells the model which tokens should be considered.

Token	Example ID	Attention mask
[CLS]	101	1
bert	14324	1
preprocessing	17463	1
[SEP]	102	1
[PAD]	0	0

A value of 1 means that the token is meaningful; a value of 0 means that the token is padding and should be ignored by attention.

Distilled models

A distilled language model is a smaller model trained to imitate a larger teacher model.



DistilBERT is designed to retain much of BERT's performance while reducing model size and inference cost.

Loading a pre-trained DistilBERT sentiment model

The notebook loads a DistilBERT model already fine-tuned on a binary sentiment classification task.

```
from transformers import DistilBertTokenizer, DistilBertForSequenceClassification
import torch

tokenizer = DistilBertTokenizer.from_pretrained(
    'distilbert-base-uncased-finetuned-sst-2-english'
)

model = DistilBertForSequenceClassification.from_pretrained(
    'distilbert-base-uncased-finetuned-sst-2-english'
)
```

Here the model already contains both a language encoder and a sentiment classification head.

Tokenizing two sentences

The notebook then prepares two example sentences.

```
sentence_a = 'i love this product, it helped me alot'
sentence_b = 'this does not work for me'

inputs = tokenizer(
    [sentence_a, sentence_b],
    return_tensors='pt',
    padding=True
)
```

The argument `return_tensors='pt'` tells the tokenizer to return PyTorch tensors.

From logits to predicted class

The model produces logits, which are raw class scores before normalization.

```
outputs = model(**inputs)
predicted_classes = torch.argmax(outputs.logits, dim=1)
```

Let $z_{i,k}$ be the logit for example i and class k. The predicted class is:

$$\hat{y}_i = \arg\max_k z_{i,k}$$

where $\hat{y}_i$ is the predicted label for example i, and $z_{i,k}$ is the unnormalized model score for class k.

Single-sentence prediction

For one sentence, the notebook extracts the predicted class and maps it to a human-readable label.

```
inputs = tokenizer(sentence_a, return_tensors='pt', padding=True)
outputs = model(**inputs)

predicted_class = torch.argmax(outputs.logits, dim=1).item()
class_labels = ['Negative', 'Positive']

print(f"The sentiment of the text is: {class_labels[predicted_class]}")
```

Using the Hugging Face pipeline

The notebook also shows the high-level `pipeline` interface.

```
from transformers import pipeline

sentiment = pipeline(
    task='sentiment-analysis',
    framework='pt',
    model='distilbert-base-uncased-finetuned-sst-2-english'
)

results = sentiment('i am in a very bad mood today')
```

The pipeline hides tokenization, model inference, logits processing, and label decoding behind a compact API.

From simple inference to custom fine-tuning

The second part of the notebook moves from using an already fine-tuned model to building a custom classifier.

The target task is BBC news classification into five categories:

`business`, `entertainment`, `sport`, `tech`, `politics`

The goal is no longer to classify sentiment, but to assign each article to its correct news category.

Loading the BBC News dataset

The notebook expects a CSV file with two columns: category and text.

```
import pandas as pd

datapath = './data/bbc-text.csv'
df = pd.read_csv(datapath)
df.head()
```

The category column contains the target label, while the text column contains the input article.

Inspecting class frequencies

Before training, the notebook checks the distribution of classes.

```
df.groupby(['category']).size().plot.bar()
```

This is an important diagnostic step. If one class is much more frequent than the others, accuracy alone may become misleading.

Tokenizing longer texts

For the news classification task, the notebook uses `bert-base-cased`.

```
tokenizer = BertTokenizer.from_pretrained('bert-base-cased')

bert_input = tokenizer(
    example_text,
    padding='max_length',
    max_length=60,
    truncation=True,
    return_tensors='pt'
)
```

The tokenizer automatically adds special tokens, pads shorter examples, and truncates longer examples.

The role of [CLS] in classification

For a classification task, the notebook focuses on the pooled representation associated with [CLS].

The intuition is:

- BERT produces a contextual vector for each token;
- the final [CLS] representation is used as a sequence-level representation;
- the classifier maps this vector into category scores.

If $h_{\texttt{CLS}} \in \mathbb{R}^{768}$ is the BERT vector for [CLS], the classification head transforms it into a vector of class scores.

Label encoding

Text labels are mapped into integers.

```
labels = {
    'business'      : 0,
    'entertainment': 1,
    'sport'         : 2,
    'tech'          : 3,
    'politics'      : 4
}
```

Neural networks do not train directly on string labels. The target category must be represented numerically.

PyTorch Dataset and DataLoader

The notebook introduces two fundamental PyTorch abstractions.

Object	Role
`Dataset`	Stores samples and labels; defines how to retrieve one item.
`DataLoader`	Creates mini-batches, shuffles data, and iterates during training.

This separation keeps preprocessing and model training modular.

Custom Dataset class

The custom dataset tokenizes every article and stores the corresponding label.

```
class Dataset(torch.utils.data.Dataset):
    def __init__(self, df):
        self.labels = [labels[label] for label in df['category']]
        self.texts = [tokenizer(text,
                                padding='max_length',
                                max_length=512,
                                truncation=True,
                                return_tensors="pt")
                      for text in df['text']]

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        return self.texts[idx], np.array(self.labels[idx])
```

Train, validation, and test split

The dataset is randomly shuffled and split into three subsets.

```
np.random.seed(112)

df_train, df_val, df_test = np.split(
    df.sample(frac=1, random_state=42),
    [int(.8 * len(df)), int(.9 * len(df))]
)
```

The proportions are approximately:

80% training, 10% validation, 10% test.

The custom BERT classifier

The notebook defines a class `BertClassifier` that inherits from `nn.Module`.

The model contains:

- a pre-trained `BertModel`;
- a dropout layer for regularization;
- a linear layer mapping a 768-dimensional BERT vector to 5 class scores;
- a ReLU activation in the notebook implementation.

The 768-dimensional input corresponds to the hidden size of BERT base.

Classifier definition

```
from torch import nn
from transformers import BertModel

class BertClassifier(nn.Module):
    def __init__(self, dropout=0.5):
        super(BertClassifier, self).__init__()
        self.bert = BertModel.from_pretrained('bert-base-cased')
        self.dropout = nn.Dropout(dropout)
        self.linear = nn.Linear(768, 5)
        self.relu = nn.ReLU()
```

The final output has dimension 5 because the BBC dataset has five classes.

Forward pass

The forward pass extracts the pooled [CLS] representation and feeds it to the classifier head.

```
def forward(self, input_id, mask):
    _, pooled_output = self.bert(
        input_ids=input_id,
        attention_mask=mask,
        return_dict=False
    )

    dropout_output = self.dropout(pooled_output)
    linear_output = self.linear(dropout_output)
    final_layer = self.relu(linear_output)

    return final_layer
```

Forward pass as a mathematical map

The classifier can be summarized as:

$$h_{\texttt{CLS}} = \mathrm{BERT}(x, m)$$

$$z = W h_{\texttt{CLS}} + b$$

$$\hat{y} = \arg\max_k z_k$$

where:

- x is the vector of input token IDs;
- m is the attention mask;
- $h_{\texttt{CLS}}$ is the pooled BERT representation;
- W and b are the classifier weights and bias;
- z_k is the score for class k;
- $\hat{y}$ is the predicted class.

Training objective

The notebook uses cross-entropy loss for multi-class classification.

$$\mathcal{L} = -\sum_{k=1}^{K} y_k \log p_k$$

where:

- $K = 5$ is the number of news categories;
- y_k is 1 if the true class is k and 0 otherwise;
- p_k is the predicted probability for class k;
- $\mathcal{L}$ is the loss minimized during training.

In PyTorch, `nn.CrossEntropyLoss()` expects raw logits as input and internally applies the softmax operation.

Training setup

The training function creates datasets, data loaders, loss function, optimizer, and device configuration.

```
train, val = Dataset(train_data), Dataset(val_data)

train_dataloader = torch.utils.data.DataLoader(
    train, batch_size=2, shuffle=True
)
val_dataloader = torch.utils.data.DataLoader(
    val, batch_size=2
)

criterion = nn.CrossEntropyLoss()
optimizer = Adam(model.parameters(), lr=learning_rate)
```

Why `zero_grad()` is needed

PyTorch accumulates gradients by default.

Therefore, before computing a new gradient on a new mini-batch, we must clear the old gradients:

$$\nabla_\theta \mathcal{L}_{\text{new batch}}$$

should not be accidentally added to previous gradients.

In the training loop this is done by:

```
model.zero_grad()
```

where θ denotes all trainable model parameters.

Training loop logic

For each epoch, the notebook performs:

1. iterate over mini-batches from the training set;
2. move inputs and labels to CPU or GPU;
3. compute model outputs;
4. compute cross-entropy loss;
5. compute gradients with backpropagation;
6. update parameters with Adam;
7. measure training accuracy and validation accuracy.

The validation set is used to monitor generalization during training.

Evaluation on the test set

After training, the model is evaluated on unseen data.

```
def evaluate(model, test_data):
    test = Dataset(test_data)
    test_dataloader = torch.utils.data.DataLoader(test, batch_size=2)

    total_acc_test = 0
    with torch.no_grad():
        for test_input, test_label in test_dataloader:
            mask = test_input['attention_mask'].to(device)
            input_id = test_input['input_ids'].squeeze(1).to(device)
            output = model(input_id, mask)
            acc = (output.argmax(dim=1) == test_label).sum().item()
            total_acc_test += acc
```

Why `torch.no_grad()` is used

During evaluation we do not update the model parameters.

Using `torch.no_grad()` tells PyTorch not to store the computation graph needed for backpropagation.

This has two benefits:

- evaluation is faster;
- memory consumption is lower.

Complete workflow

The notebook shows the full pipeline: from raw text to tokenized tensors, from BERT representations to class scores, and from class scores to final predictions.

Important implementation caveats

There are some points worth discussing with students.

- The notebook uses `ReLU` after the final linear layer. For `CrossEntropyLoss`, the usual choice is to return raw logits without a final activation.
- The tokenizer example uses `max_length=60`, while the full dataset uses `max_length=512`. This difference should be explicitly explained.
- The batch size is very small, which is convenient for demonstration but inefficient for serious training.
- Full fine-tuning of BERT may be slow on CPU and is much more practical on GPU.

Possible improvements for a teaching version

A more robust teaching notebook could add:

- a clearer separation between inference and fine-tuning examples;
- explicit train, validation, and test metrics at each epoch;
- a confusion matrix and per-class precision, recall, and F1-score;
- a comparison between frozen BERT and fully fine-tuned BERT;
- reproducible environment specifications for `torch`, `transformers`, `accelerate`, and `protobuf`.

Key takeaways

- BERT-like models transform text into contextual token representations.
- The tokenizer is not a minor detail: it defines the numerical input seen by the model.
- For classification, the `[CLS]` representation is commonly used as a sequence-level feature.
- DistilBERT is useful when we need faster and lighter inference.
- Fine-tuning adapts a general language model to a specific supervised task.

References mentioned in the notebook

- Hugging Face Transformers documentation.
- Hugging Face BERT model documentation.
- Ruben Winastwan, *Text Classification with BERT in PyTorch*.
- Rayyan Shaikh, *Mastering BERT: A Comprehensive Guide from Beginner to Advanced in Natural Language Processing*.
- Jay Alammar, *A Visual Guide to Using BERT for the First Time*.